

# Vector Similarity: Cosine, Dot, and L2, Worked Out by Hand

What a retriever actually computes when it ranks a query against documents — three small vectors, every dot product, norm, cosine, and distance counted by hand.

**What this document does.** Module 2.5 treats the retriever as a black box: a `Runnable` that takes a query and returns a ranked `list[Document]`. This document opens that box for one operation — the *scoring* step. We take a 3-dimensional query embedding  $q = (1, 0, 1)$  and three document embeddings, and compute, by hand, the three numbers a vector store can rank by: the **dot product**, the **cosine similarity**, and the **L2 (Euclidean) distance**. We then derive the exact identity that makes cosine and L2 rank the same way for normalized embeddings. Every number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent.

**Where this sits.** This is **Topic 1 of Module 2.5** — the “2-minute RAG recap” names embeddings, a vector store, and a retriever interface, then declares the internals out of scope. This document is that one missing page of internals: *how a vector store decides which document is closest*. Its companions: the RAG course (embedding models, chunking, vector-store ops) and Topic 2 of this module (BM25 lexical scoring — the *other* half of the hybrid retrieval the recap mentions).

**Concepts covered, in order:** the embedding as a vector · the dot product  $a \cdot b = \sum a_i b_i$  · the L2 norm  $|a|$  · cosine similarity as a normalized dot product · squared Euclidean distance  $|a - b|^2 = |a|^2 + |b|^2 - 2a \cdot b$  · the identity  $|a - b|^2 = 2(1 - \cos \theta)$  for unit vectors · why cosine ignores magnitude and dot does not · why embedding search defaults to cosine.

*Internal learning material. Numbers are reproducible from Appendix A. v1.0*

---

## 1 The setup: one query, three documents, small enough to count

A retriever embeds the query and every document into the same vector space, then ranks documents by how “close” they are to the query. “Close” is a number, and there are three common choices for it. To see how they differ — and where they agree — we need vectors small enough to compute on paper. Real embeddings have 768 or 1,536 dimensions; the mechanics are *identical* at three.

**The vectors.** The query and documents are 3-dimensional. They are chosen round so every sum is followable, and chosen to expose the one fact that trips people up:  $d_1$  and  $d_2$  point in the *same direction* but have different lengths.

| Symbol | Meaning         | Vector      | Note                                  |
|--------|-----------------|-------------|---------------------------------------|
| $q$    | query embedding | $(1, 0, 1)$ | the thing we rank against             |
| $d_1$  | document 1      | $(1, 0, 1)$ | identical to $q$                      |
| $d_2$  | document 2      | $(2, 0, 2)$ | same direction as $q$ , twice as long |
| $d_3$  | document 3      | $(0, 1, 0)$ | orthogonal to $q$                     |

**Notation.** We write  $a \cdot b$  for the dot product,  $|a| = \sqrt{a \cdot a}$  for the length (L2 norm),  $\theta$  for the angle between two vectors, and  $|a - b|$  for the straight-line (Euclidean) distance between their tips. A vector is *unit-normalized* if  $|a| = 1$ .

### Intuition

An embedding turns “meaning” into a *direction* in space. Two documents about the same thing point roughly the same way; an unrelated document points elsewhere. The three scores below are three different rulers for “same way”: cosine measures *only* the angle, the dot product measures angle *scaled by length*, and L2 measures the gap between the tips. They agree on what is closest far more often than they disagree — but the cases where they disagree are exactly the ones worth understanding.

## 2 Step 1 — the dot product

The dot product multiplies the vectors coordinate-by-coordinate and sums:

$$a \cdot b = \sum_{i=1}^n a_i b_i. \quad (1)$$

For our query against each document:

$$q \cdot d_1 = (1)(1) + (0)(0) + (1)(1) = 2, \quad q \cdot d_2 = (1)(2) + (0)(0) + (1)(2) = 4,$$

$$q \cdot d_3 = (1)(0) + (0)(1) + (1)(0) = 0.$$

So by raw dot product the ranking is  $d_2(4) > d_1(2) > d_3(0)$ . ✓ Notice already something suspicious:  $d_2$  “wins,” but  $d_2$  is just  $d_1$  scaled up by two. It is not *more relevant*; it is merely *longer*. Hold that thought.

### Mini-example — this operation in isolation

The dot product is also how we get a vector’s own length. Setting  $b = a$  in Equation (1) gives  $a \cdot a = \sum a_i^2$ , and the L2 **norm** is its square root:  $|a| = \sqrt{a \cdot a}$ . For our vectors:  $|q| = \sqrt{1+0+1} = \sqrt{2} \approx 1.4142$ ,  $|d_1| = \sqrt{2}$ ,  $|d_2| = \sqrt{4+0+4} = \sqrt{8} \approx 2.8284$  (exactly  $2|q|$ , since  $d_2 = 2d_1$ ), and  $|d_3| = \sqrt{0+1+0} = 1$ . These four lengths are the only extra ingredient cosine and L2 need.

## 3 Step 2 — cosine similarity (the dot product, de-scaled)

The dot product confuses “points the same way” with “is long.” Cosine similarity removes the length by dividing out both norms. It is exactly  $\cos \theta$ , the cosine of the angle between the two vectors:

$$\cos \theta = \frac{a \cdot b}{|a| |b|}. \quad (2)$$

Working each document, with  $|q| = \sqrt{2}$ :

$$\cos(q, d_1) = \frac{2}{\sqrt{2} \cdot \sqrt{2}} = \frac{2}{2} = 1.0, \quad \cos(q, d_2) = \frac{4}{\sqrt{2} \cdot \sqrt{8}} = \frac{4}{\sqrt{16}} = \frac{4}{4} = 1.0,$$

$$\cos(q, d_3) = \frac{0}{\sqrt{2} \cdot 1} = 0.0.$$

Here is the punchline.  $d_1$  and  $d_2$  **both score cosine** = 1.0 — the maximum. Their dot products differed (2 vs. 4), but once we divide out the length, they are *identical*: same direction, angle zero, perfect match. And  $d_3$ , at right angles to  $q$ , scores cosine 0. ✓

### Intuition

Cosine asks one question: *ignoring how big the vectors are, do they point the same way?* That is precisely what we want from a document search. A document that says the same thing twice as emphatically (a longer chunk, more repeated words) is not twice as relevant — and cosine, unlike the dot product, refuses to reward it for mere length.  $\cos \theta$  ranges from 1 (identical direction) through 0 (perpendicular, unrelated) to  $-1$  (opposite).

## 4 Step 3 — L2 distance, and why $|a - b|^2 = |a|^2 + |b|^2 - 2a \cdot b$

The third ruler is straight-line distance: how far apart are the *tips* of the two vectors? Squared, that is  $|a - b|^2 = \sum_i (a_i - b_i)^2$ . Expanding the square term-by-term reveals it is built from the same dot products we already have:

$$\begin{aligned} |a - b|^2 &= \sum_i (a_i - b_i)^2 = \sum_i (a_i^2 - 2a_i b_i + b_i^2) \\ &= \underbrace{\sum_i a_i^2}_{|a|^2} - 2 \underbrace{\sum_i a_i b_i}_{a \cdot b} + \underbrace{\sum_i b_i^2}_{|b|^2} = |a|^2 + |b|^2 - 2(a \cdot b). \end{aligned} \quad (3)$$

This is just the law of cosines in disguise. Plugging in our precomputed norms and dot products:

$$|q - d_1|^2 = 2 + 2 - 2(2) = 0, \quad |q - d_2|^2 = 2 + 8 - 2(4) = 2, \quad |q - d_3|^2 = 2 + 1 - 2(0) = 3.$$

So  $|q - d_1| = 0$ ,  $|q - d_2| = \sqrt{2} \approx 1.4142$ ,  $|q - d_3| = \sqrt{3} \approx 1.7321$ . ✓ By L2 the ranking (nearest first) is  $d_1 (0) < d_2 (\sqrt{2}) < d_3 (\sqrt{3})$ .

### Watch out

Look carefully:  $d_1$  and  $d_2$  had *identical* cosine (1.0), but their L2 distances *differ* — 0 versus  $\sqrt{2}$ . Raw L2, like the dot product, is sensitive to magnitude:  $d_2$  sits twice as far out along the same ray, so its tip is genuinely farther from  $q$ 's tip, even though it points the same way. Cosine says “same direction, perfect match”; raw L2 says “farther away.” They only *rank the same* once we strip magnitude out — which is the next step, and the reason production stores normalize.

## 5 Step 4 — the identity: for unit vectors, L2 ranking = cosine ranking

The three rulers disagreed because two of them see magnitude. Remove magnitude — normalize every vector to length 1 — and two of them *collapse into one*. Set  $|a| = |b| = 1$  in Equation (3). Then  $|a|^2 = |b|^2 = 1$ , and the dot product  $a \cdot b$  is, by Equation (2), exactly  $\cos \theta$  (the denominator is 1). Substituting:

$$\boxed{|a - b|^2 = 1 + 1 - 2 \cos \theta = 2(1 - \cos \theta) \quad (\text{for } |a| = |b| = 1).} \quad (4)$$

Read this equation as a statement about *ranking*. The right side,  $2(1 - \cos \theta)$ , is a **strictly decreasing** function of  $\cos \theta$ : the bigger the cosine (more similar), the smaller the squared distance (closer). So sorting unit vectors by ascending L2 distance gives *exactly* the same order as sorting by descending cosine — one is the inverse of the other.

### Mini-example — this operation in isolation

Check the identity on  $q$  and  $d_3$ , which are already convenient. Normalize:  $\hat{q} = q/\sqrt{2} = (0.707, 0, 0.707)$  and  $\hat{d}_3 = d_3/1 = (0, 1, 0)$ . Their cosine is still 0 (normalizing never changes direction). Left side:  $|\hat{q} - \hat{d}_3|^2 = (0.707)^2 + (-1)^2 + (0.707)^2 = 0.5 + 1 + 0.5 = 2$ . Right side:  $2(1 - 0) = 2$ . ✓ Equal, as promised. And for  $d_1, d_2$  (both normalize to  $\hat{q}$ , cosine 1): left side 0, right side  $2(1 - 1) = 0$ . ✓ Once normalized, the magnitude difference that split their L2 distances vanishes — they become the *same* unit vector.

This is why a vector store can offer “cosine” and “L2” as interchangeable metrics: if it normalizes embeddings at index time (most do), the two produce the identical top- $K$ . The retriever returns the same ranked `list[Document]` either way.

## 6 Step 5 — the three rulers, side by side

Collecting every number computed above into one table makes the pattern unmistakable:

| Pair     | $q \cdot d$ | $ d $      | $\cos \theta$ | $ q - d $  | rank verdict           |
|----------|-------------|------------|---------------|------------|------------------------|
| $q, d_1$ | 2           | $\sqrt{2}$ | 1.0           | 0          | identical              |
| $q, d_2$ | 4           | $\sqrt{8}$ | 1.0           | $\sqrt{2}$ | same meaning, longer   |
| $q, d_3$ | 0           | 1          | 0.0           | $\sqrt{3}$ | unrelated (orthogonal) |

The three columns rank the documents three ways:

- **Dot product:**  $d_2 > d_1 > d_3$  — crowns the *longest*, not the most relevant.  $d_2$  wins only because it is twice as big.
- **Cosine:**  $d_1 = d_2 > d_3$  —  $d_1$  and  $d_2$  *tie* at 1.0, because they point the same way. Magnitude is invisible.
- **Raw L2:**  $d_1 < d_2 < d_3$  — ranks  $d_1$  strictly ahead of  $d_2$ ; like the dot product it still sees magnitude, just with the opposite sign of preference.

Cosine is the odd one out — and the one we usually want — precisely because it is the only ruler that calls  $d_1$  and  $d_2$  equally relevant.

## 7 Why embedding search defaults to cosine

Embedding vectors carry a quiet bias: their *length* tends to track surface properties like token count, word frequency, or how “confident” the encoder was — not the meaning. A long document chunk and a short one about the identical topic can land on the same ray from the origin but at very different distances out. That is exactly the  $d_1$  versus  $d_2$  situation.

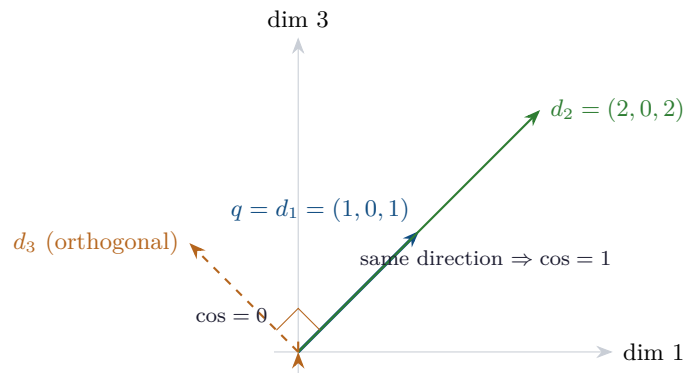
**Dot product** would reward the longer chunk for being longer — usually not what a search wants. **Raw L2** would penalize it for sitting farther out — also a magnitude artifact. **Cosine** throws

magnitude away and keeps only the direction, which is where the meaning lives. So semantic search defaults to cosine (or, equivalently, normalize-then-use-L2, by Equation (4)). This is the “dense” similarity behind the retriever in Module 2.5: the query and each chunk become vectors, and the top- $K$  chunks are the ones with the highest cosine to the query.

### Intuition

Direction is meaning; length is mostly bookkeeping. Two sentences that say the same thing point the same way no matter how many words they spend doing it. Cosine is the tool that listens to the *what* and ignores the *how loudly*. Topic 2’s BM25 is the complementary tool — it scores exact word overlap instead of direction — and a hybrid retriever runs both, because some queries are won on meaning and some on the literal token.

## 8 The geometry, drawn



$q$ ,  $d_1$ , and  $d_2$  all lie on one ray from the origin — same *direction*, so cosine = 1 for every pair among them, regardless of how far out each tip sits.  $d_3$  leaves at a right angle: cosine 0. Cosine reads the *angle* between rays; the dot product and raw L2 also read how far along the ray each tip is. Normalizing slides every tip onto the unit circle, after which L2 and cosine tell the same story (Equation (4)).

## 9 Similarity cheat-sheet

|                      |                                                              |
|----------------------|--------------------------------------------------------------|
| Dot product          | $a \cdot b = \sum_i a_i b_i$                                 |
| L2 norm (length)     | $ a  = \sqrt{a \cdot a} = \sqrt{\sum_i a_i^2}$               |
| Cosine similarity    | $\cos \theta = \frac{a \cdot b}{ a   b } \in [-1, 1]$        |
| Squared L2 distance  | $ a - b ^2 =  a ^2 +  b ^2 - 2(a \cdot b)$                   |
| Unit-vector identity | $ a - b ^2 = 2(1 - \cos \theta)$ when $ a  =  b  = 1$        |
| Ranking equivalence  | sort $\uparrow$ L2 = sort $\downarrow$ cosine (unit vectors) |
| What cosine ignores  | magnitude / length (so: direction = meaning)                 |

## 10 An honest word about these numbers

The vectors here are 3-dimensional with integer coordinates, picked so every dot product and norm is followable on paper — exactly as the cost-by-hand document used round token sizes no one would ship. A real embedding is a dense float vector of 768 or 1,536 dimensions, and a real corpus has millions of them behind an approximate-nearest-neighbour index, not three you can read off a table. What is *not* illustrative is the structure: the dot product is always  $\sum a_i b_i$ ; cosine is always that divided by the two norms; squared L2 always expands to  $|a|^2 + |b|^2 - 2a \cdot b$ ; and for unit vectors  $|a - b|^2$  is always exactly  $2(1 - \cos \theta)$ , so the cosine and L2 rankings are always inverses. Change the dimension or the corpus and the specific scores move; those identities — and the reason semantic search prefers cosine — do not.

## Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the dot products, norms, cosines, L2 distances, and the unit-vector identity check; change the vectors to explore your own.

```

1  # vector_similarity.py -- reproduces every number in "Vector Similarity, by Hand".
2  import math
3
4  def dot(a, b): return sum(x * y for x, y in zip(a, b))
5  def norm(a): return math.sqrt(dot(a, a))
6  def cosine(a, b): return dot(a, b) / (norm(a) * norm(b))
7  def l2sq(a, b): return sum((x - y) ** 2 for x, y in zip(a, b))
8  def l2(a, b): return math.sqrt(l2sq(a, b))
9  def unit(a): n = norm(a); return tuple(x / n for x in a)
10
11 q = (1, 0, 1)
12 d1 = (1, 0, 1) # identical to q
13 d2 = (2, 0, 2) # same direction, twice as long
14 d3 = (0, 1, 0) # orthogonal to q
15
16 print(f"|q|={norm(q):.4f} |d1|={norm(d1):.4f} "
17       f"|d2|={norm(d2):.4f} |d3|={norm(d3):.4f}")
18
19 # --- the three rulers for each document ---
20 for name, d in [("d1", d1), ("d2", d2), ("d3", d3)]:
21     print(f"q-{name}: dot={dot(q, d)} cos={cosine(q, d):.4f} "
22           f"L2^2={l2sq(q, d)} L2={l2(q, d):.4f}")
23
24 # --- identity check: |a-b|^2 == 2(1-cos) for UNIT vectors ---
25 print("identity |unit(q)-unit(d)|^2 vs 2(1-cos):")
26 for name, d in [("d1", d1), ("d2", d2), ("d3", d3)]:
27     lhs = l2sq(unit(q), unit(d))
28     rhs = 2 * (1 - cosine(q, d)) # cosine is scale-invariant
29     print(f"q-{name}: lhs={lhs:.4f} rhs={rhs:.4f}")
30
31 # Expected output:
32 # |q|=1.4142 |d1|=1.4142 |d2|=2.8284 |d3|=1.0000
33 # q-d1: dot=2 cos=1.0000 L2^2=0 L2=0.0000
34 # q-d2: dot=4 cos=1.0000 L2^2=2 L2=1.4142
35 # q-d3: dot=0 cos=0.0000 L2^2=3 L2=1.7321
36 # identity |unit(q)-unit(d)|^2 vs 2(1-cos):
37 # q-d1: lhs=0.0000 rhs=0.0000
38 # q-d2: lhs=0.0000 rhs=0.0000
39 # q-d3: lhs=2.0000 rhs=2.0000

```

---

— *end of document* —